SmartPOS — Initial Requirements Document

Project title: SmartPOS: Web-Based Point of Sale and Inventory Management System
Course: Software Construction and Development Lab — 5th Semester
Instructor: Engr. Muhammad Umer Haroon
Version: 0.1.0
Status: Initial requirements; subject to refinement through development
Date: 19 October 2026

1. Group Members

Member	GitHub username	Initial responsibility
Asma Saqib	[github-username]	Project lead, repo maintainer, documentation
Sidra Noor	[github-username]	Flask backend / database
Muskan Iqbal	[github-username]	Frontend / checkout interface

2. Problem Statement

Small shops need a practical way to manage products, stock, billing, and sales records. Manual billing and stock tracking can be slow and may cause calculation errors, inaccurate inventory, and difficulty reviewing past sales. Some traditional POS solutions depend on dedicated scanners, receipt printers, or other specialized equipment.

SmartPOS aims to provide a locally runnable web application that allows authorized users to manage products and process cash sales using a normal laptop. It will support product selection through manual search or QR codes scanned using a browser-accessible webcam, without requiring dedicated POS hardware.

3. Project Description

SmartPOS is a Python Flask web application with an SQLite database and a browser-based interface. It will support separate administrator and cashier roles. Administrators manage users, products, inventory, and shop settings. Cashiers search for or scan products, prepare a cart, complete cash transactions, and view permitted transaction details.

The first release will be designed to run on one computer using a local Flask server and a web browser. Product QR codes will encode a product identifier (SKU), while current product details and prices will be retrieved from the database.

4. Objectives

- Develop a working web-based POS and inventory application using Python and Flask.
- Store products, users, sales, and sale items persistently in SQLite.
- Provide role-based access for administrators and cashiers.
- Support manual product search and webcam QR scanning.
- Calculate totals and cash change accurately.
- Update inventory safely when a sale succeeds.
- Provide sales history, basic reporting, and printable receipts.
- Apply collaborative Git practices, modular design, testing, and maintainable documentation.

5. Intended Users

- **Administrator / shop owner:** manages users, products, stock, settings, and reports.
- **Cashier:** searches or scans products, prepares bills, accepts cash, and completes sales.
- **Project evaluator / developer:** reviews implementation, requirements, tests, and Git history.

6. Initial Functional Requirements

ID	Requirement	Priority
FR-01	The system shall allow registered users to log in and log out securely.	Must
FR-02	The system shall distinguish administrator and cashier permissions.	Must
FR-03	Administrators shall add, edit, search, and deactivate products.	Must
FR-04	Product records shall include a unique SKU, name, category, price, and stock quantity.	Must
FR-05	The system shall generate a QR code for a product SKU.	Must
FR-06	Cashiers shall find products using manual search.	Must
FR-07	Cashiers shall scan a product QR code through a supported browser webcam.	Must
FR-08	The system shall retrieve current product details from SQLite after a SKU is scanned or searched.	Must
FR-09	Cashiers shall add items to a cart, change quantities, and remove items.	Must
FR-10	The system shall calculate the subtotal and any configured discount or tax consistently.	Must
FR-11	The system shall accept cash received, reject insufficient cash, and calculate change due.	Must
FR-12	On successful checkout, the system shall save a sale and its line items and update stock.	Must
FR-13	Checkout updates shall be atomic: if a transaction fails, partial sale and stock changes shall be rolled back.	Must
FR-14	The system shall produce a receipt that can be printed from the browser.	Should
FR-15	Authorized users shall view sales history and transaction details according to their role.	Must
FR-16	Administrators shall view basic sales summaries and low-stock products.	Should
FR-17	Administrators shall configure basic shop details and a low-stock threshold.	Could
FR-18	Administrators shall export relevant reports to CSV.	Could
FR-19	The system shall provide a useful error message for unknown QR codes and unavailable cameras.	Must
FR-20	The system shall allow manual product search when camera scanning is unavailable.	Must

7. Initial Non-Functional Requirements

ID	Requirement
NFR-01 Usability	Common cashier tasks should be accessible through a clear, consistent browser interface.
NFR-02 Security	Passwords shall be stored as hashes; restricted routes shall enforce authorization on the server.
NFR-03 Data integrity	Database constraints and transactions shall protect sales and stock consistency.
NFR-04 Reliability	Product and transaction records shall persist after application restart.
NFR-05 Portability	The initial release shall run on a supported Python environment and modern browser on a laptop.
NFR-06 Hardware independence	The application shall not require a dedicated barcode scanner, cash register, or receipt printer. A compatible webcam is optional because manual search is available.
NFR-07 Offline operation	Core local operations shall not require cloud services or an internet connection once required dependencies are installed.

ID	Requirement
NFR-08 Maintainability	Backend routes, business logic, templates, and tests shall be organized into understandable modules.
NFR-09 Testability	Core business rules, including login permissions, product lookup, cash validation, and checkout, shall have repeatable tests.
NFR-10 Privacy	The camera shall be used only after browser permission is granted; camera access shall not be needed for manual search.

8. Product Backlog

Priorities: **Must** = essential for a usable first release; **Should** = important if time permits; **Could** = optional enhancement.

ID	Backlog item / user story	Priority	Acceptance criteria
PB-01	As a user, I can log in and log out.	Must	Valid credentials work; invalid credentials are rejected.
PB-02	As an administrator, I can manage users and roles.	Must	Only authorized users can access admin functions.
PB-03	As an administrator, I can manage products and stock.	Must	Valid products persist and duplicate SKUs are rejected.
PB-04	As an administrator, I can generate product QR labels.	Must	QR encodes the correct unique SKU.
PB-05	As a cashier, I can search products manually.	Must	Matching active products and current prices are displayed.
PB-06	As a cashier, I can scan product QR codes using a webcam.	Must	A valid scan resolves to the correct database product; unknown codes show an error.
PB-07	As a cashier, I can build and edit a cart.	Must	Quantities and totals update correctly; invalid quantities are rejected.
PB-08	As a cashier, I can complete a cash sale.	Must	Insufficient cash is rejected; change is correct; sale and stock update succeed together.
PB-09	As a cashier, I can view/print a receipt.	Should	Receipt contains receipt number, items, quantities, prices, total, cash, and change.
PB-10	As an administrator, I can review sales and low stock.	Should	Dashboard values are derived from database records.
PB-11	As an administrator, I can export reports.	Could	CSV output matches selected records.
PB-12	As an administrator, I can configure shop details.	Could	Settings persist and appear where relevant.
PB-13	As a developer, I can run automated tests.	Must	Core tests run with documented commands.
PB-14	As a team, we maintain clear documentation and traceable Git history.	Must	README, requirements, meaningful commits, and release tags are maintained.

9. Initial Assumptions

- The first version is intended for one shop operating on one computer.
- The local Flask application and SQLite database run on the same computer.
- Product QR labels can be printed on ordinary paper or displayed on a screen.
- Each product has a unique SKU used as its QR identifier.

- A compatible webcam and browser permission are needed for webcam scanning; manual search remains available.
- Payments are cash-only in the first release. The system records the cash received; it does not connect to a bank or payment gateway.
- The team will use GitHub for shared version control and issue/task tracking.

10. Constraints and Known Limitations

- The application will not require dedicated POS hardware, but webcam scanning requires a compatible camera and browser support.
- Browser camera access is normally available on `localhost` or a secure HTTPS context; other deployment configurations may require HTTPS.
- Initial operation is local and single-shop; multi-branch and cloud synchronization are out of scope.
- Card payments, online payments, accounting integrations, and customer loyalty features are out of scope for the first release.
- The initial requirements may change after feedback, prototyping, and testing.
- Real deployment security, backups, and concurrent multi-user operation across multiple devices may need additional work beyond the first lab release.